

第八章 注意力机制：大模型为什么能懂上下文？

8.1 注意力的直觉：打分与加权平均

8.1.1 一个词，两种意思

“我喜欢吃苹果。”

“苹果发布新手机。”

第一句里的“苹果”是水果，第二句里的“苹果”是公司。你之所以能立刻区分，是因为你看到了它周围的词：“吃”把“苹果”拉向了水果的含义，“发布”和“新手机”把它拉向了科技公司。每个词的精确含义，取决于它和句子里其他词之间的关系。

这件事对人类来说毫不费力。但对模型来说，这是一个真实的难题。

我们先回忆一下，一句话是怎么输入到模型里的。以“我喜欢吃苹果”为例，它首先被拆成若干个 token。为了方便，我们假设它被拆成了四个：“我”“喜欢”“吃”“苹果”（实际的分词结果可能不同，但这不影响理解原理）。每个 token 在词表里有一个编号，比如“苹果”的编号是 100，“吃”是 87，“喜欢”是 203，“我”是 56。这些编号本身没有任何含义：你不能说 203 就比 100 “更大”或者“更好”，它们只是身份证号码。

但模型没法直接拿整数做计算。所以接下来，每个编号会被转成一个向量，也就是一串数字组成的列表。这个转换过程叫做词嵌入（Embedding），本质上就是拿着编号去一张巨大的表里查，查出来的一行数字就是这个词的向量。以一个 7B（70 亿参数）的模型为例，每个词的向量通常有 4096 个维度。你可以想象成一串包含 4096 个数字的列表。不同模型的维度不同，但这里的关键点不在于具体多少维，而在于另一件事：

同一个词，不管出现在什么句子里，查出来的向量是一模一样的。

“苹果”在词表里只有一个编号，查出来永远是同一个向量。不管你把它放进“我喜欢吃苹果”还是“苹果发布新手机”，Embedding 给出的向量没有任何区别。

这就是问题所在。在 Embedding 这一步结束之后，模型眼中的每个词都是一座孤岛。“苹果”只知道自己是“苹果”，不知道自己在这句话里到底是水果还是公司。而自然语言里到处都是这样的情况，比如“打”可以是打人、打电话、打酱油；“过”可以是走过、过年、过分；甚至“他”这个字，在不同句子里指的是完全不同的人。

如果模型不能根据上下文来调整每个词的含义，它对语言的理解就永远停留在查字典的水平：给每个词一个固定的解释，不管上下文是什么。

所以模型需要一个机制：让每个词能“看到”句子里的所有其他词，根据看到的内容来更新自己的向量。更新之后，“苹果”在水果语境里的向量和在科技公司语境里的向量就会变得不同。

这就是注意力机制要做的事情。接下来我们就在“我喜欢吃苹果”这个句子上，一步一步看清它到底是怎么运作的。

8.1.2 每个词给所有词打一个分

注意力机制的核心操作只有两步：打分和加权平均。

我们直接进入例子。

假设“我喜欢吃苹果”经过 Embedding 之后，四个词各自得到了一个 2 维的向量（真实模型是几千维，这里用 2 维是为了能手算。维度高了只是数字多了，计算逻辑完全一样）：

词	向量
我	[0, 1]
喜欢	[1, 0]
吃	[1, 2]
苹果	[1, 1]

这四个向量就是四个词此刻的全部身份信息。但它们还是孤立的：每个词只包含自己的信息，不包含任何来自上下文的信息。

现在，注意力机制要做的第一步是：让每个词对句子里的所有词（包括自己）打一个分。分数越高，代表那个词对我越重要。

整个句子有四个词，每个词都要对所有四个词打分，一共产生 16 个分数。我们先不纠结这些分数是怎么算出来的（那是下一节的核心内容），先把它们当作一个已知的结果来用。下面这张表展示的就是这 16 个分数。假设模型经过某种计算之后，得到了这样一组数字：

	我	喜欢	吃	苹果
我	3	4	1	1
喜欢	3	2	3	3
吃	1	2	2	4
苹果	1	2	4	3

怎么读这张表？每一行代表这个词在看谁，数字越大代表越关注。每一列代表被谁看。

比如最后一行是“苹果”的视角。苹果给“吃”打了 4 分，是全行最高的。给自己打了 3 分，给“喜欢”2 分，给“我”只有 1 分。这很符合直觉：在“我喜欢吃苹果”这句话里，决定“苹果”含义的关键词就是“吃”。有了“吃”，你马上知道这里的“苹果”是食物。

再看第三行“吃”的视角。“吃”给“苹果”打了 4 分，全行最高。这也合理：对于“吃”来说，最重要的信息是“吃什么”，答案就是“苹果”。

注意到了吗？“吃”和“苹果”互相给了对方最高分。“吃”觉得“苹果”最重要，“苹果”也觉得“吃”最重要。这种双向的高分，意味着它们会深度交换信息，最终各自的向量里都会融入大量对方的信息。

再看第一行，也就是“我”的视角。“我”给“喜欢”打了 4 分，最高。这也很直觉：“我喜欢”是一个紧密的主谓结构，“我”最需要关注的就是“我在做什么”。

你可以从这张表里看出一个有意思的规律：不同的词关注的对象完全不同。“我”最关注“喜欢”，“吃”最关注“苹果”，“苹果”最关注“吃”。每个词都在从自己的角度审视这句话，寻找对自己最重要的信息。

需要提醒一下：这组分数是我为了讲解方便而设计的，刻意让它符合人类对语义关系的直觉。真实模型算出来的分数不一定这么合理。现在先用这组数字把计算流程走完。

8.1.3 从原始分数到注意力权重

我们把目光集中在“苹果”这一行。它对四个词打出的原始分数是 $[1, 2, 4, 3]$ 。

但光有这四个数字还不够。接下来我们要用这组分数对所有词的向量做加权平均。这就要求权重加起来必须等于 1，否则加权平均在数学上说不通。 $1 + 2 + 4 + 3 = 10$ ，加起来不是 1。

怎么把一组任意大小的数字变成一组加起来恰好等于 1 的权重，同时保持原来的大小关系？前面讲损失函数的时候你已经见过这个操作了：Softmax。它对每个分数取 e 的指数，然后除以所有指数的总和。

算一下：

- $e^1 \approx 2.72$
- $e^2 \approx 7.39$
- $e^4 \approx 54.60$
- $e^3 \approx 20.09$

$$\text{总和} = 2.72 + 7.39 + 54.60 + 20.09 = 84.80$$

四个权重分别是：

- 我： $2.72/84.80 \approx 0.03$
- 喜欢： $7.39/84.80 \approx 0.09$
- 吃： $54.60/84.80 \approx 0.64$
- 苹果： $20.09/84.80 \approx 0.24$

可以验证： $0.03 + 0.09 + 0.64 + 0.24 = 1.00$ 。

经过 Softmax 之后，原始分数 $[1, 2, 4, 3]$ 变成了 $[0.03, 0.09, 0.64, 0.24]$ 。大小关系保持了：“吃”依然最大，“我”依然最小，但数字被重新归一化到了一组合法的权重。

这组权重就是“苹果”的**注意力权重**。它告诉我们：在更新“苹果”的向量时，“吃”贡献 64% 的权重，“苹果”自己贡献 24%，“喜欢”贡献 9%，“我”只贡献 3%。

面试考点 · Softmax 在注意力中的作用不只是“归一化”

常见问法：“为什么注意力机制用 Softmax 而不是简单地除以总和来归一化？”

典型错误：回答“为了让权重加起来等于 1”。这只能说对了一半。简单地用每个分数除以所有分数的总和，也能让权重加到 1，但注意力偏偏用了 Softmax。

正确思路：Softmax 相比简单的除以总和，有两个关键区别。

第一，Softmax 里有指数函数，它会放大分数之间的差距。原始分数里“最大的”和“第二大的”可能只差一两分，但经过指数之后差距会被显著拉开，让权重分布更“尖锐”，注意力更集中。用一个小例子就能看到这一点：原始分数 $[1, 2, 4, 3]$ 里最大值 4 和第二大值 3 只差 1，但 Softmax 之后 0.64 vs 0.24 差了将近三倍。

第二，Softmax 保证所有权重都是正数（因为指数函数的值域是正数），这对加权平均很重要。如果权重出现负数，加权平均的物理含义就变了，不再是融合信息，而是减去信息。简单的除以总和做不到这个保证（如果原始分数有负数，归一化后权重也会有负数）。

8.1.4 用注意力权重做加权平均

现在我们拿着这组权重 $[0.03, 0.09, 0.64, 0.24]$ ，对四个词的向量做加权平均。

回忆一下四个词的向量：我 $[0, 1]$ ，喜欢 $[1, 0]$ ，吃 $[1, 2]$ ，苹果 $[1, 1]$ 。

苹果的新向量 $= 0.03 \times [0, 1] + 0.09 \times [1, 0] + 0.64 \times [1, 2] + 0.24 \times [1, 1]$

逐维度计算：

- 第一维： $0.03 \times 0 + 0.09 \times 1 + 0.64 \times 1 + 0.24 \times 1 = 0 + 0.09 + 0.64 + 0.24 = 0.97$
- 第二维： $0.03 \times 1 + 0.09 \times 0 + 0.64 \times 2 + 0.24 \times 1 = 0.03 + 0 + 1.28 + 0.24 = 1.55$

苹果的新向量 $= [0.97, 1.55]$ 。

和原始向量 $[1, 1]$ 对比一下。第一维几乎没变（ $1 \rightarrow 0.97$ ），但第二维大幅增长（ $1 \rightarrow 1.55$ ）。为什么第二维涨了这么多？因为“吃”的向量是 $[1, 2]$ ，它的第二维特别大（是 2），而“吃”又拿到了 0.64 的权重，在加权平均中贡献了 $0.64 \times 2 = 1.28$ 这么大的一块。“吃”的信息被大量灌入了“苹果”的新向量里。

这就是注意力计算完成的事情：“苹果”的向量不再只包含自己的信息，它现在融入了整句话的信息，而且“吃”的信息占了最大的份额。在模型的语言经验里，“吃”这个动作和食物、水果紧密关联。当“吃”的信息深度融入“苹果”的向量之后，“苹果”的新向量就自然地偏向于水果的方向。

面试考点 · 所有分数一样大时，注意力在做什么

常见问法：“如果一个词对句子里所有词的注意力分数都相等，输出会是什么？这意味着什么？”

正确思路：如果所有分数都相等，Softmax 之后每个词的权重也相等（比如 4 个词各 0.25），加权平均的结果就是所有词向量的简单算术平均。这意味着这个词没有从上下文中提取到任何差异化的信息；它只是把所有词的信息均匀地混在了一起。这种情况相当于注意力机制失效了，退化成了一个简单的平均池化操作。

面试官考这道题，是想看你是否理解注意力的价值恰恰在于分配不均匀，也就是有选择地关注某些词、忽略另一些词。如果注意力均匀了，选择就消失了。

8.1.5 换一个句子，一切都变了

现在来看“苹果发布新手机”这句话。简化成三个词：“苹果”“发布”“新手机”。

至关重要的一点：**苹果的初始向量仍然是 $[1, 1]$ ，和上一句完全一样。**因为 Embedding 查表是固定的，同一个词查出来永远是同一个向量。

但这一次，苹果面对的上下文完全变了。假设三个词的向量是：

词	向量
苹果	$[1, 1]$
发布	$[2, 1]$
新手机	$[2, 0]$

假设经过打分之后，苹果对三个词的原始分数是：苹果 2 分，发布 4 分，新手机 3 分。“发布”拿到了最高分。我们直觉上也能感受到，“发布”是决定“苹果”在这句话里含义的关键词，有了“发布”，你马上知道这是科技公司，不是水果。至于模型为什么也会给出这样的打分，下一节讲 Q、K、V 的时候会解释。

Softmax 归一化：

- $e^2 \approx 7.39$
- $e^4 \approx 54.60$
- $e^3 \approx 20.09$

总和 = 82.08

三个权重：

- 苹果： $7.39/82.08 \approx \mathbf{0.09}$
- 发布： $54.60/82.08 \approx \mathbf{0.67}$
- 新手机： $20.09/82.08 \approx \mathbf{0.24}$

加权平均：

苹果的新向量 = $0.09 \times [1, 1] + 0.67 \times [2, 1] + 0.24 \times [2, 0]$

- 第一维： $0.09 \times 1 + 0.67 \times 2 + 0.24 \times 2 = 0.09 + 1.34 + 0.48 = \mathbf{1.91}$
- 第二维： $0.09 \times 1 + 0.67 \times 1 + 0.24 \times 0 = 0.09 + 0.67 + 0 = \mathbf{0.76}$

苹果的新向量 = $[1.91, 0.76]$ 。

原始向量是 $[1, 1]$ ，第一维从 1 跳到了 1.91，第二维从 1 降到了 0.76。“发布”的向量 $[2, 1]$ 的第一维很大，它以 0.67 的权重强力拉动了苹果的向量，把它拽向了第一维方向。

现在我们把两个句子的结果放在一起看：

	“我喜欢吃苹果”	“苹果发布新手机”
苹果的初始向量	$[1, 1]$	$[1, 1]$
最关注的词	吃（权重 0.64）	发布（权重 0.67）
苹果更新后的向量	$[0.97, 1.55]$	$[1.91, 0.76]$

图 8.1 把这种变化画了出来：左图里“吃”的柱子一枝独秀，右图里高峰跳到了“发布”。

同一个“苹果”，同一个初始向量 $[1, 1]$ ，经过一轮注意力计算之后，在两个句子里变成了两个方向截然不同的向量。第一句里，第二维从 1 涨到了 1.55；第二句里，第二

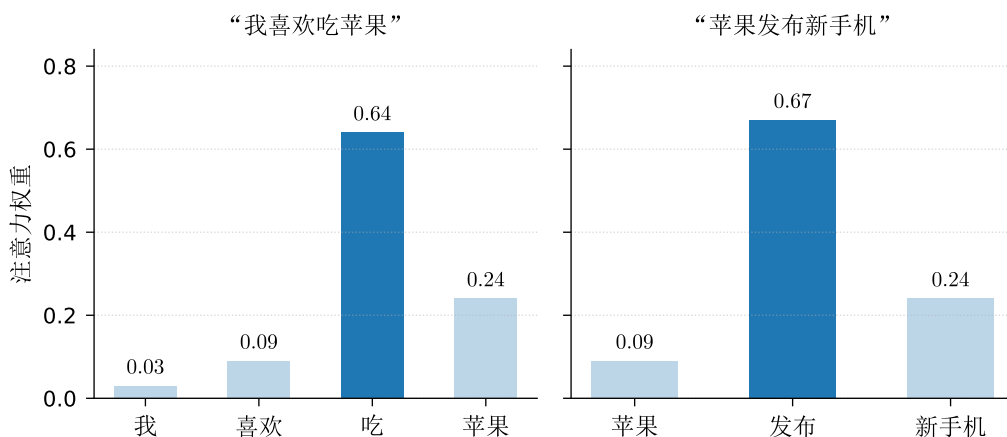


图 8.1: “苹果”在两个不同句子中的注意力权重分布。左图中“吃”独占 0.64 的权重，右图中权重重心转移到了“发布”（0.67）。同一个词，面对不同的上下文，注意力的分配方式完全不同。

维反而从 1 降到了 0.76。两个新向量朝着几乎相反的方向偏移了。图 8.2 把这种分叉画在了同一个坐标系里，三个箭头从同一个原点出发，却指向了完全不同的位置。

区别不是来自“苹果”这个词本身，它的初始向量完全一样。区别来自它周围的词。不同的上下文产生了不同的注意力权重，不同的注意力权重导致加权平均的结果不同，最终同一个词在两个句子里获得了完全不同的向量表示。

如果把注意力机制拿掉会怎样？模型就只能用 Embedding 查表给出的那个固定向量 [1,1] 来代表“苹果”。不管上下文是什么，“苹果”在模型眼里永远长一个样，永远分不清水果和公司。注意力机制让每个词的向量从“固定的字典释义”变成了“动态的、随上下文变化的表示”。这是质的提升。

8.1.6 所有词同时在做这件事

上面的演示里，我们一直在以“苹果”的视角来描述整个过程：苹果给所有词打分，苹果做加权平均，苹果更新向量。但实际上，这套操作是句子里的所有词**同时**做的。“我”也在同时对四个词打分、做加权平均、更新自己的向量；“喜欢”也在同时做；“吃”也在同时做。它们之间没有先后顺序，不需要等谁先算完。

回想一下前面讲 RNN 的时候，RNN 必须一个词一个词按顺序处理：先处理“我”，产生一个中间状态传给“喜欢”，“喜欢”处理完再传给“吃”，“吃”处理完再传给“苹果”。像流水线上的工人，前一个做完了后一个才能开始。一句话有 100 个词，RNN 就要走 100 步。GPU 上那成千上万个计算核心大部分时间都在闲着。

注意力机制没有这个限制。每个词看所有词、打分、做 Softmax、加权平均，这些操作对于每个词来说是独立的，可以完全并行。100 个词的句子，所有词的注意力可以在同一时刻完成。GPU 的大规模并行能力终于有了用武之地。

但并行带来了一个新问题：既然所有词同时处理，那模型怎么知道谁在前、谁在后？回头看我们的打分过程：“苹果”对“吃”打分的时候，只看它们的向量是否相似，根本没有用到“吃在苹果前面”这个位置信息。也就是说，如果把“我喜欢吃苹果”打乱成“苹果喜欢吃我”，每个词的向量没变，同一对词之间的打分也不会因为位置交换而变化，只是分数表的行列跟着词的位置一起换了。但这两句话的意思显然不同。

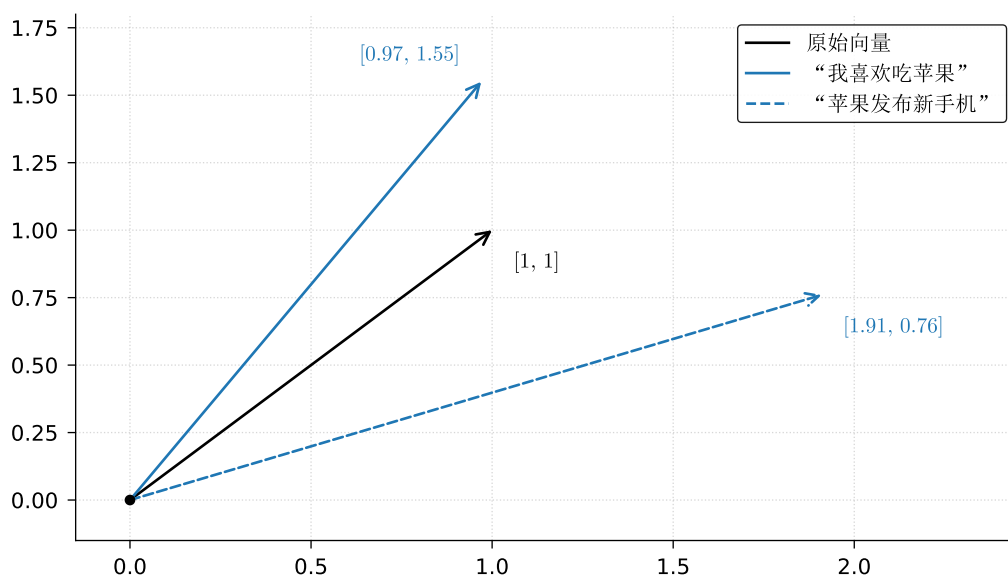


图 8.2: “苹果”的向量在两个不同语境下的分叉。三个箭头从同一个原点出发: 黑色实线是 Embedding 查表得到的初始向量 $[1, 1]$, 蓝色实线是在“我喜欢吃苹果”中经过注意力计算后的新向量, 蓝色虚线是在“苹果发布新手机”中的新向量。同一个起点, 被不同的上下文拉向了截然不同的方向。

这个问题确实存在, 后面讲位置编码的时候会解决它。你现在只需要知道: 注意力机制本身不区分词序, 需要另一个组件来补上位置信息。

8.1.7 注意力分数不是训练出来的参数

到这里, 你已经走完了注意力的核心流程。在往下走之前, 有一个区分非常重要, 值得单独拿来说清楚, 因为“注意力权重”这个词很容易和你之前学过的“权重”混在一起。中文里“权重”这个词被用来指至少两种完全不同的东西, 一种是训练出来的固定参数, 一种是注意力算出来的动态比例。英文里至少还能靠加不加 attention 来区分, 中文语境下纯靠上下文猜。

你在前面见过的“权重”, 比如线性层的权重矩阵、Embedding 查找表, 都是模型在训练过程中学到的参数。训练结束后它们就固定下来, 推理的时候每次都同一组值。

注意力机制里的“注意力权重”是完全不同的东西。我们刚才算出来的那些数字 $[0.03, 0.09, 0.64, 0.24]$ 不是训练出来的参数, 而是在每一次前向传播的时候, 由当前输入的数据实时计算出来的。换一句话输入, 这些数字就完全变了。我们刚刚看到的就是最好的证明: 同一个模型处理“我喜欢吃苹果”和“苹果发布新手机”, 用的是同一组模型参数, 但“苹果”的注意力分布完全不同, 一个把 64% 给了“吃”, 另一个把 67% 给了“发布”。

这是注意力机制和之前所有操作最大的不同。在此之前你学过的每一种操作, 比如线性变换、Embedding 查表, 都是“参数固定, 输入变, 输出变”。输入决定了计算的对象, 但计算的方式(参数)是不变的。注意力机制第一次出现了“连计算的方式本身都由输入决定”的情况。谁跟谁平均、平均多少, 不是写死在模型参数里的, 而是由当前这句话的内容实时决定的。

你可以这么理解: 模型面对每一句话, 都会临时搭建一个信息交换网络。“苹果”应

该从“吃”那里获取大量信息，从“我”那里获取很少；但换一句话，这个网络就被拆掉重建。“苹果”变成要从“发布”那里获取大量信息。

那模型真正训练的参数在哪里？在打分的方式里。我们这一节跳过了“分数是怎么算出来的”这个问题，下一节会讲到，打分依赖三个叫 Q、K、V 的矩阵，它们的参数才是训练出来的。你可以这么理解：模型训练的不是“苹果应该关注吃”这种具体的对应关系，而是一套通用的“如何打分”的能力。这套能力训练好之后，拿到任何一句话，都能算出合理的注意力分布。

顺带一提，我们这一节里把每句话内部的词互相打分、互相加权平均的操作，有一个专门的名字叫**自注意力**（self-attention）。“自”的意思是这句话在看它自己内部的词，区别于后面会遇到的另一种情况：一句话的词去看另一句话的词。在目前阶段，你碰到的都是自注意力。

8.1.8 几个容易掉进去的坑

注意力机制的原理不算复杂，但围绕它有几个非常常见的误解。在你直觉还在成形的阶段，趁早纠正掉，比后面再改要省力得多。

第一个坑：看到注意力热力图就觉得“模型在关注那个词，所以才做出了那个预测”。

你去搜注意力机制的科普文章，几乎每篇都会画一张热力图，颜色越深代表注意力权重越高。很多人看到热力图就觉得找到了模型决策的“原因”——“看，模型关注了‘吃’，所以它知道苹果是水果。”

图 8.3 展示的就是这种热力图的样子：颜色越深，表示这一行的词越关注那一列的词。图中右下角黑框标出的两格对应前文的直觉例子：“吃”高度关注“苹果”，“苹果”也高度关注“吃”。

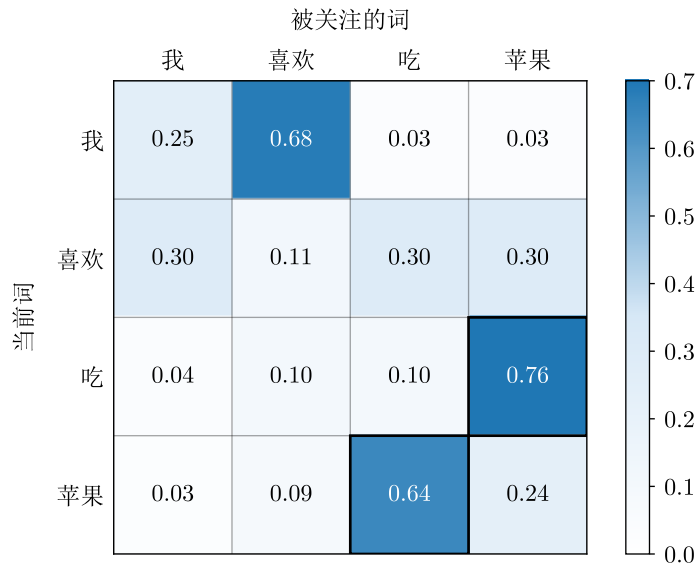


图 8.3: “我喜欢吃苹果”这句话的一张注意力热力图。颜色越深，表示对应位置的注意力权重越高。它能展示这一轮注意力计算里的权重分布，但不能直接当成模型最终决策的因果解释。

但事情没那么简单。注意力权重只是模型整个计算流程中一个中间环节的数字。一

个词经过注意力计算得到新向量之后，后面还有好几步加工操作才会到达最终的输出（这些操作在后续章节会一一介绍）。而且模型通常有几十层，每一层都有自己的注意力权重。拿其中某一层的热力图说“这就是模型做出这个预测的原因”，就像一条一百米长的工厂流水线，你跑去中间某一个环节拍了张照片，就说“最终产品是因为这个环节才变成这样的”。中间的步骤太多了，因果关系远没有那么直接。你盯着花花绿绿的色块看下午，大概率什么结论也得不出来。先查数据质量、再看 loss 曲线，比盯热力图靠谱得多。

更反直觉的是：研究者实际观察大模型的注意力分布时发现，很多注意力头（后面会讲到，一层里有多个并行的注意力模块）把大量权重分配给了句子的第一个 token，或者标点符号，或者“的”“了”“是”这种高频功能词，不管这些词的语义是什么。这就引出了下一个坑。

第二个坑：以为每个词会主要关注和自己“语义相关”的词。

直觉上你会觉得“苹果”应该主要关注“吃”或者“发布”这种语义上明确相关的词。我们上面那个例子也确实是这样的，但那是我为了讲清楚概念而刻意设计的。

在真实训练出来的大模型里，注意力分布经常不符合人类的直觉。比如，你可能会发现“苹果”把很大一部分注意力给了“的”这个字，或者给了句子的第一个 token，不管那个 token 是什么词。

为什么？因为注意力权重是为最终的任务目标服务的。对于大语言模型来说，这个目标通常是“预测下一个词”。模型在训练中学到的“谁该关注谁”，是让预测最准确的分配方式，不是让人类觉得最合理的分配方式。模型可能发现，把一些注意力分配给句首的标点或者功能词，能帮助它完成某种全局信息的汇总。这种策略人类看了觉得奇怪，但模型用了确实预测更准。

第三个坑：以为注意力机制是 Transformer 发明的。

注意力机制的想法最早出现在 2014 到 2015 年的机器翻译工作中。当时它是作为 RNN 的一个辅助模块被引入的。当时 RNN 在翻译长句子时，到后面几步就已经“忘”了开头的内容，注意力机制被加进来让模型在生成每个词的时候能回头看看原文，决定当前应该重点参考原文的哪个位置。那个阶段，RNN 是主角，注意力只是一个帮手。

Transformer 的贡献不是发明注意力机制，而是做了一个更大胆的决定：**把 RNN 整个扔掉，只用注意力**。“Attention Is All You Need”——Transformer 论文的标题说的就是这件事。在那之前，没有人觉得注意力可以独当一面。Transformer 证明了它不但可以，而且比有 RNN 的时候做得更好。

第四个坑：说注意力机制在理解语言。

这个措辞非常常见，很多从业者在日常讨论中也会这么说。但如果你把我们刚才做的计算过程回放一遍：算分数、做 Softmax、加权平均，每一步都是纯粹的数值运算。算分数就是两组数字相乘再相加，Softmax 就是取指数再除以总和，加权平均就是乘以权重再加起来。单独看每一步，一个计算器就能完成，里面没有任何理解的魔法。

注意力机制之所以表现得好像在理解，是因为这套简单的数值操作在几十层的堆叠和海量文本的训练之后，涌现出了看起来像理解的行为。区分“机制本身做了什么”和训练之后涌现了什么，是建立正确技术直觉的关键一步。

8.1.9 一个需要提前知道的代价

注意力计算要求每个词都和所有词算一次分数。四个词就是 $4 \times 4 = 16$ 次计算，100 个词就是 $100 \times 100 = 10000$ 次，1000 个词就是 100 万次。计算量随着序列长度的增长是平方级别的：序列长度翻一倍，计算量翻四倍。

在短句子里这完全不是问题。但当你处理一篇几千个词的长文章，或者一段几个 token 的对话历史时，这个平方增长就会变成一个很现实的瓶颈，不只是算得慢，还特别吃显存。后来出现的各种“高效注意力”方案，很大一部分工作就是在想办法缓解这个平方复杂度。这个话题我们后面会再碰到，这里先留个印象。

实战避坑：序列长度翻倍，显存不是翻倍而是翻四倍

注意力的计算量是序列长度的平方，但在真实工程里这个“平方”带来的最直接后果不是“算得慢”，而是显存直接爆掉。你的模型在 2048 token 上微调跑得好好的，产品经理说“我们要支持 4096 的上下文”，你心想长度翻一倍、显存大概也翻一倍，把 batch size 砍一半应该就行。结果一跑直接 OOM。

因为注意力那一步要存一个“序列长度 $\times$ 序列长度”的分数矩阵，长度从 2048 到 4096，这个矩阵从 $2048^2 \approx 400$ 万变成了 $4096^2 \approx 1600$ 万，涨了四倍，不是两倍。你砍一半 batch size 只省了一倍的量，远远不够。碰到这种情况，要么用 FlashAttention 之类的显存优化方案（后面会讲到），要么老老实实算一下显存预算再动手。

8.1.10 自检清单

1. 为什么 Embedding 给出的固定向量不够用？注意力机制解决了什么问题？
固定 Embedding 让同一个词在不同句子里总是同一个初始向量，没法根据上下文变化。注意力机制让每个词去看句子里的其他词，把相关信息混进来，得到带上上下文的新向量。
2. 注意力机制的两步核心操作分别是什么？每一步的输入和输出各是什么？
第一步是打分并归一化，把目标词和其他词的相关程度变成注意力权重；第二步是加权求和，用这些权重把其他词的向量合成目标词的新向量。
3. 同一个“苹果”在“我喜欢吃苹果”和“苹果发布新手机”里，经过注意力计算后得到的新向量相同吗？为什么不同？
不同。因为它身边的上下文词不一样，注意力权重也会不一样，最后加权混合出来的新向量自然不同。
4. “注意力权重”和模型的“权重参数”有什么本质区别？换一句话输入，注意力权重会变吗？模型的权重参数会变吗？
注意力权重是模型根据当前输入临时算出来的，会随着句子变化；模型的权重参数是训练学到的参数，推理时不会因为换一句话就改变。
5. 注意力计算的计算量和序列长度之间是什么关系？
普通自注意力需要让序列里每个位置都和其他位置计算关系，计算量大约随序列长度平方增长，也就是 $O(n^2)$ 。

8.1.11 Python 交互实验室

正文里我们用手算走完了注意力的三步操作，但手算只能处理两三个词的迷你句子。配套资源包中的 `attention_intuition_demo.py` 把这套计算完整地实现了出来，并且会自动画出注意力权重的柱状图和向量分叉的二维坐标图。这里的脚本是一个“点积近似版”补充实验，会用向量点积重新生成打分，而不是完全复用正文里的预设分数表；它的目的不是提前讲完 QKV，而是让你看到“相似度变化会改变注意力权重”这件事。运行之后，你会看到“苹果”在两个语境下的注意力分布和最终向量被并排放在一起。

最值得尝试的一件事：打开代码顶部的参数配置区，把“吃”的向量从 $[1, 2]$ 改成 $[0, 1]$ ，让它和“苹果”不那么相似，然后重新运行，观察“吃”的注意力权重下降了多少、“苹果”的新向量又偏向于哪个方向。你还可以试着自己编一个全新的句子（比如三个词、每个词二维），看看注意力会把目标词的向量拉向什么位置。改几轮参数之后，“打分越高的词对新向量影响越大”这件事就会从一个你知道的结论，变成一个你看到过的事实。

8.2 Q、K、V：注意力分数到底是怎么算出来的

8.2.1 分数到底是怎么来的

“我喜欢吃苹果”这句话，经过注意力机制之后，“苹果”给“吃”打了最高分，“吃”的信息被大量灌入“苹果”的新向量里。整个过程很清楚：打分、Softmax 归一化、加权平均。但有一个关键环节被跳过了：那张分数表里的数字，到底是怎么算出来的？

前面给出的分数是直接列在表里的，当时说“先不纠结这些分数是怎么算出来的”。现在是时候把这个黑箱打开了。

要给两个词打一个分，衡量它们之间的相关程度，我们需要一种运算，能接受两个向量作为输入，输出一个数字。这个数字越大，代表两个向量越匹配。学线性代数的时候你见过一个运算恰好能做这件事：**点积**。

点积的计算非常简单：两个向量对应位置的数字相乘，然后把所有乘积加起来，得到一个数字。比如 $[1, 2]$ 和 $[3, 4]$ 的点积就是 $1 \times 3 + 2 \times 4 = 3 + 8 = 11$ 。再比如 $[1, 0]$ 和 $[0, 1]$ 的点积是 $1 \times 0 + 0 \times 1 = 0$ 。

点积有一个很好的性质：两个向量的方向越一致，点积越大；方向垂直，点积为零；方向相反，点积为负。

那最直接的想法就是：拿两个词的向量直接算点积，把结果当作它们之间的注意力分数。我们来试试看，用之前那个例子里的向量算一算，看看效果怎么样。

“我喜欢吃苹果”，四个词的 Embedding 向量（2 维，和前面用的一样）：

词	向量
我	$[0, 1]$
喜欢	$[1, 0]$
吃	$[1, 2]$
苹果	$[1, 1]$

现在我们用点积给每个词对每个词打分。四个词对四个词，一共 16 个点积，排成一张表：